

Introduction to Neural Network

Abdullah Saihan

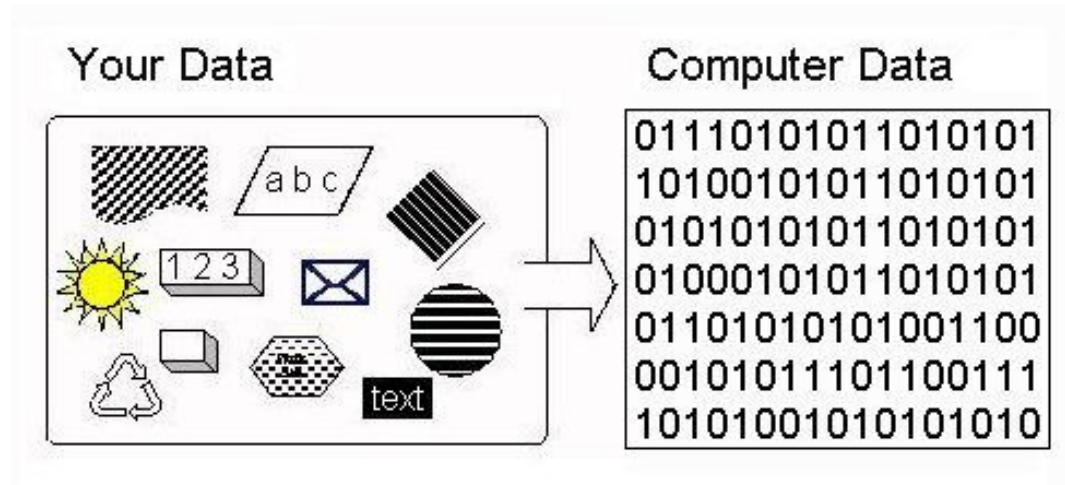
Artificial Intelligence Engineer



Table of Contents

1. Representing Data with Numbers
2. Perceptron and Neuron
3. Decision Boundary
4. Logic Gates using Perceptron
5. Neural Network

Representing data using numbers



Color
Green
Red
Black
Green

Label
encoding



Encoding
1
2
3
1

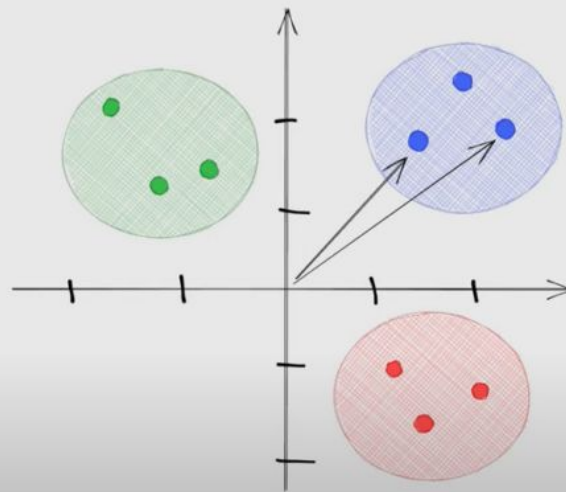
*unique labels

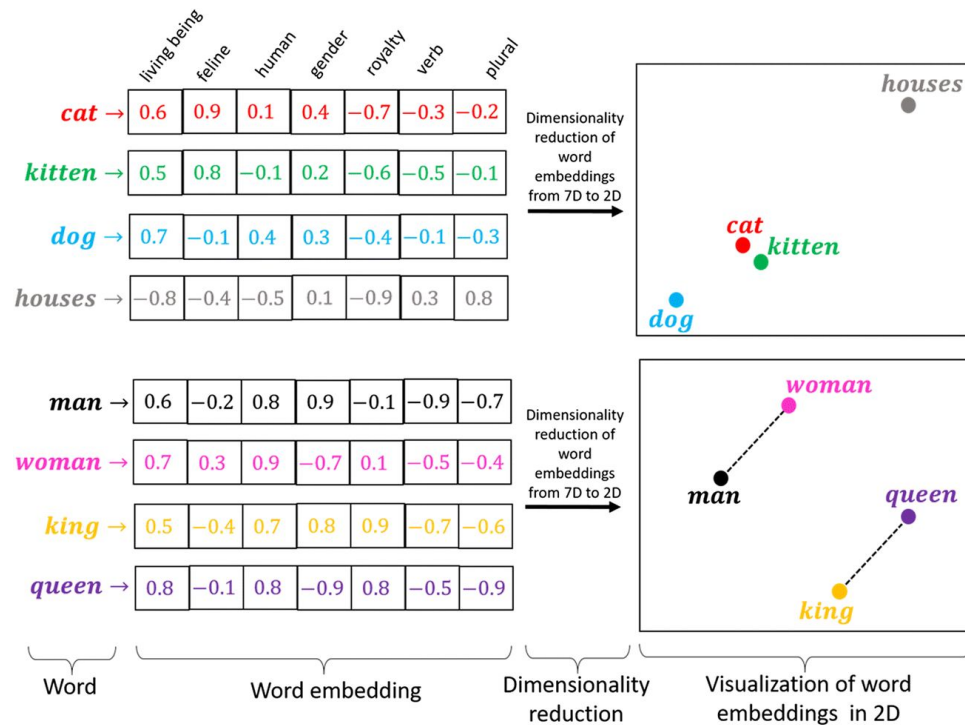
vector embeddings (2D example)

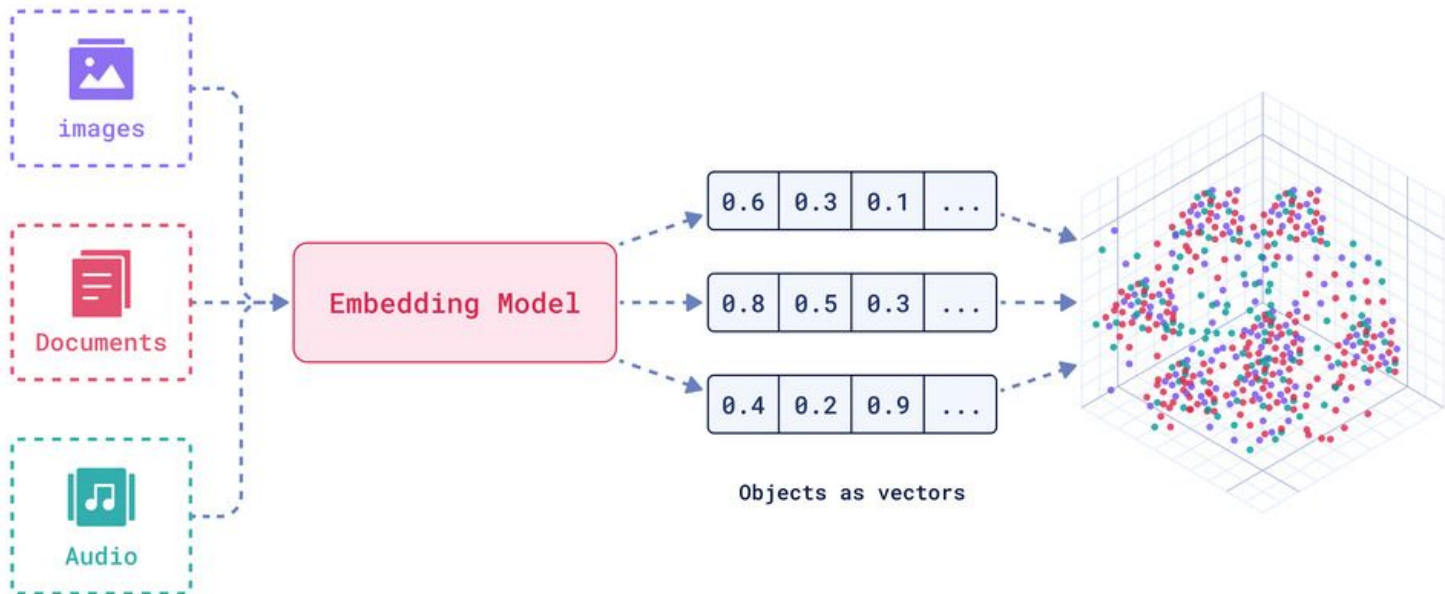
king
man
woman
apple
banana
orange
football
golf
tennis



[4 5]
[3 3.5]
[5 3]
[2.5 -2]
[2.5 -3]
[4 -2.5]
[-3 4]
[-1.5 3]
[-2 2]







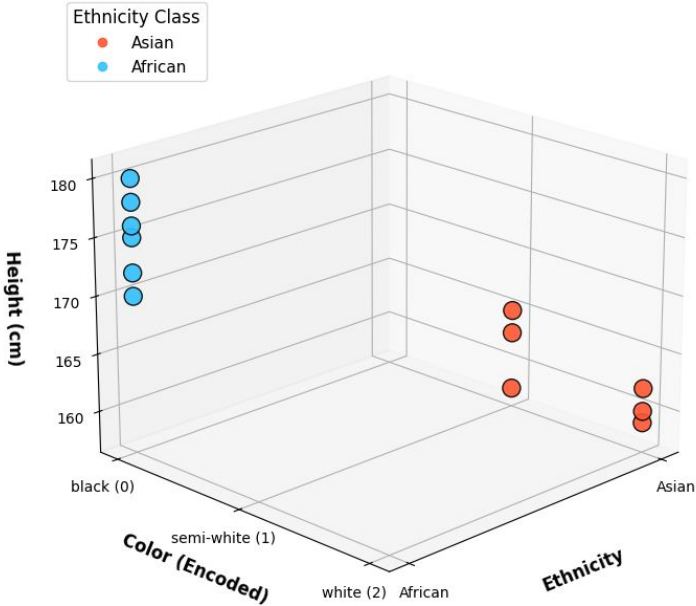
1. Original Data Table

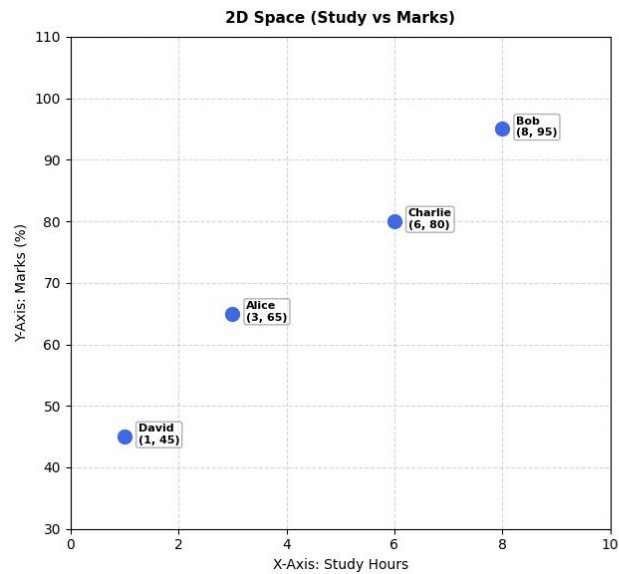
id	ethnicity	color	height_cm
1	Asian	white	162
2	Asian	semi-white	158
3	Asian	semi-white	165
4	Asian	white	160
5	Asian	semi-white	163
6	Asian	white	159
7	African	black	175
8	African	black	180
9	African	black	172
10	African	black	178
11	African	black	170
12	African	black	176

2. Color Label Encoding

id	ethnicity	color	color_encoded	height_cm
1	Asian	white	2	162
2	Asian	semi-white	1	158
3	Asian	semi-white	1	165
4	Asian	white	2	160
5	Asian	semi-white	1	163
6	Asian	white	2	159
7	African	black	0	175
8	African	black	0	180
9	African	black	0	172
10	African	black	0	178
11	African	black	0	170
12	African	black	0	176

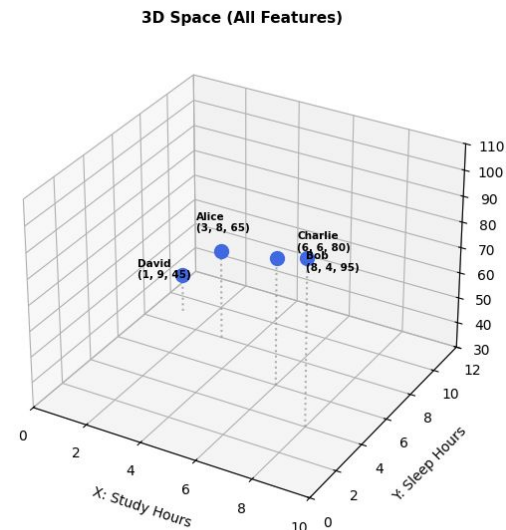
3. 3D Plot (Colored by Ethnicity)

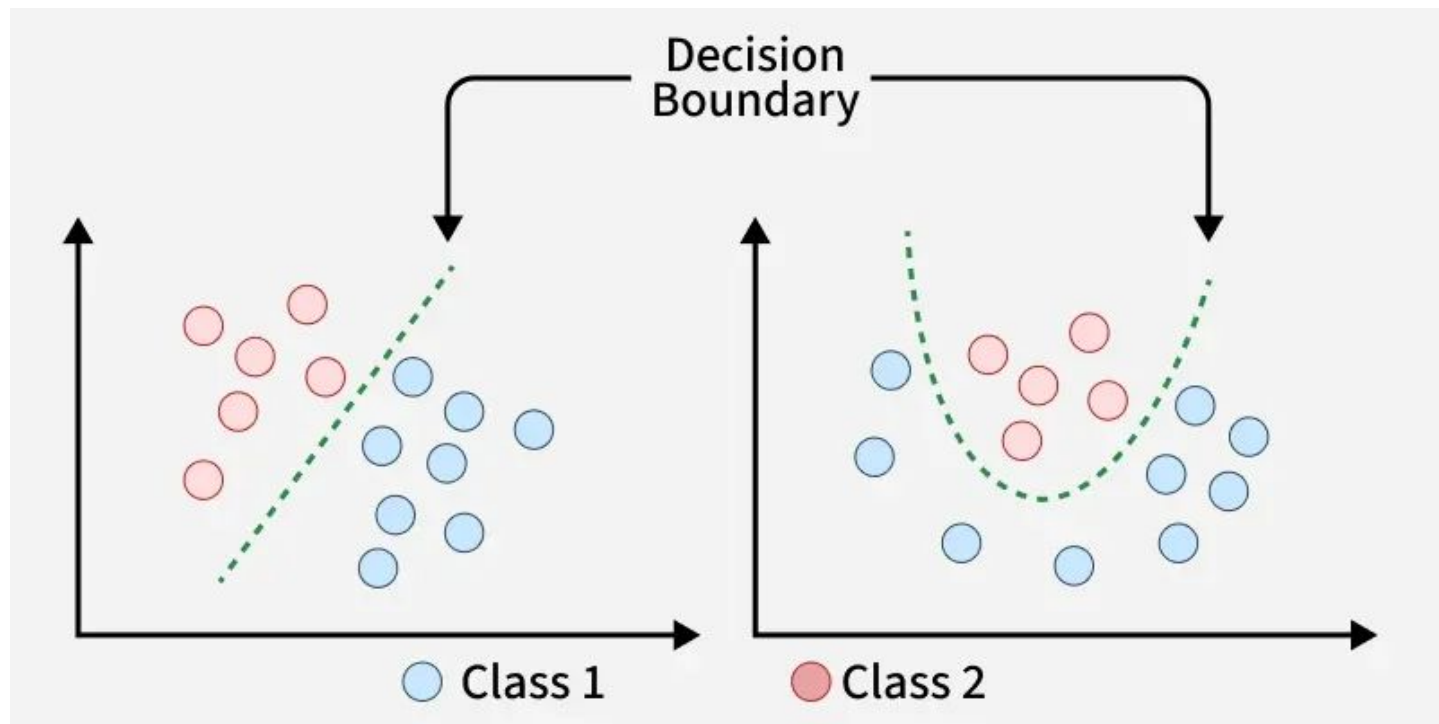




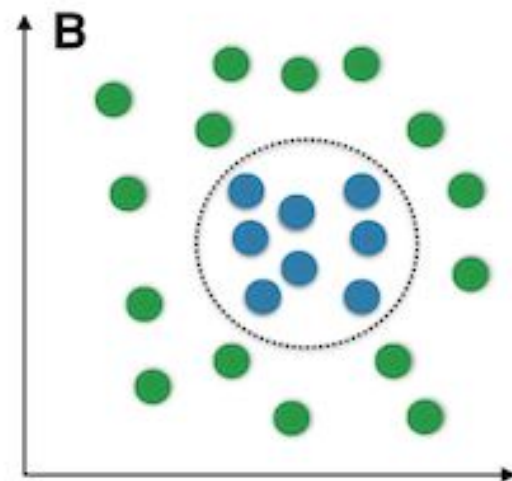
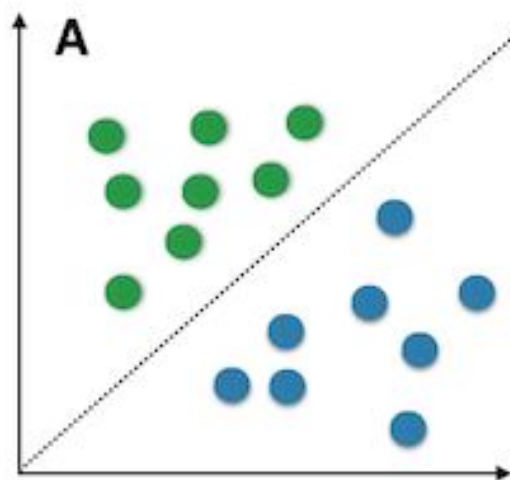
Numerical Dataset

Student	Study Hours (X)	Sleep Hours (Y)	Marks (Z)
Alice	3	8	65%
Bob	8	4	95%
Charlie	6	6	80%
David	1	9	45%

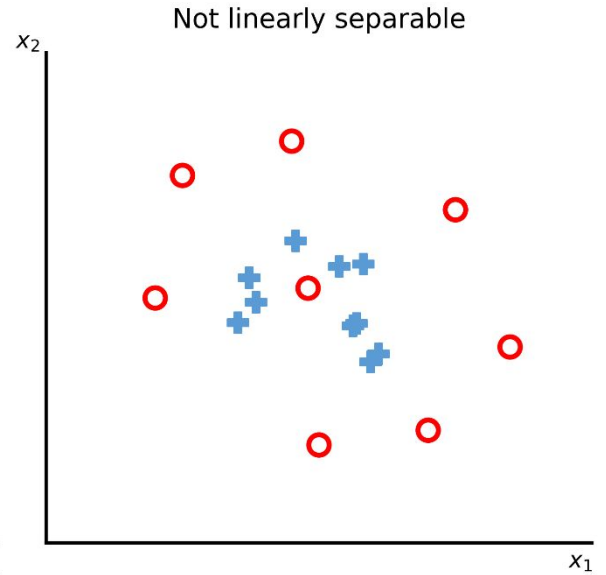
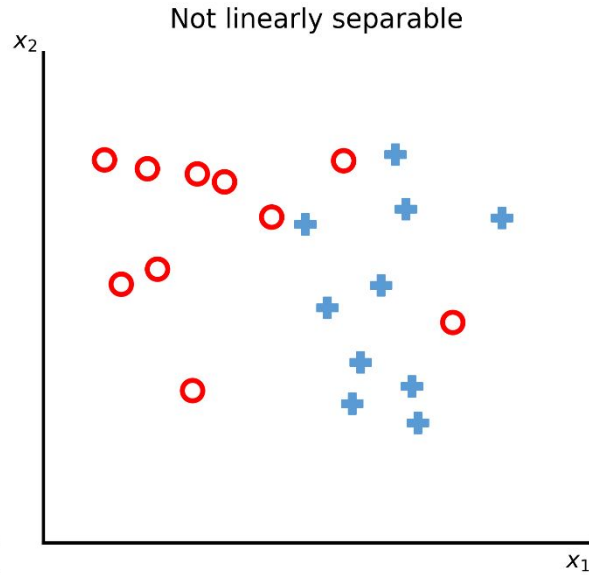
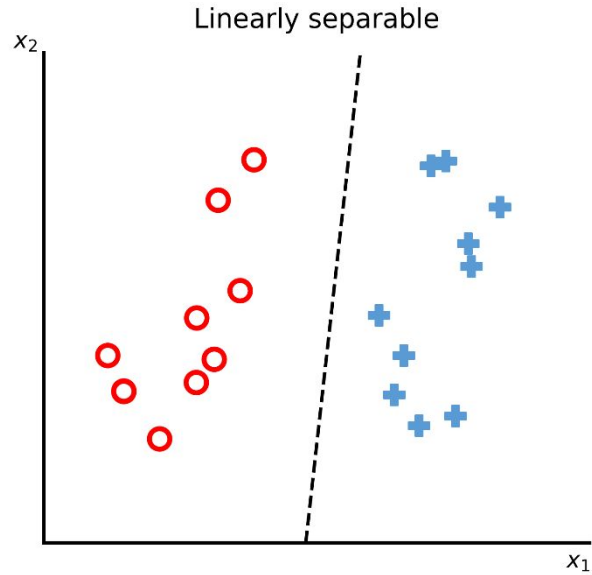




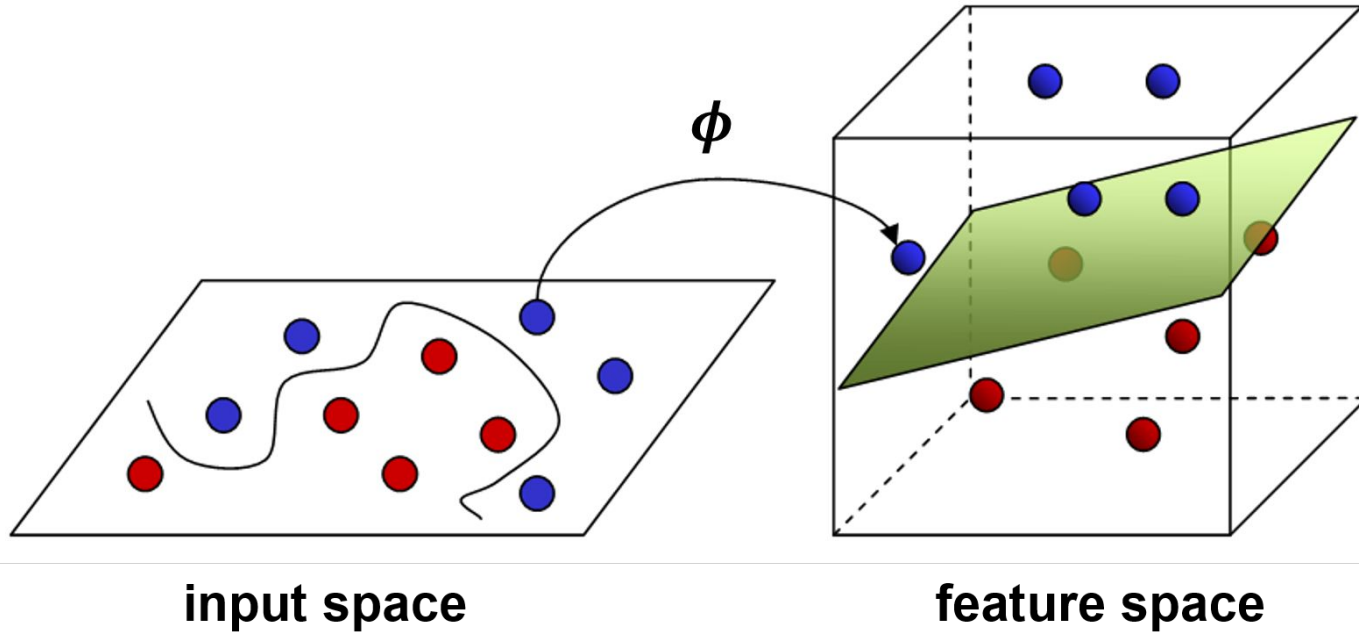
Linear vs. nonlinear problems



Linear Separability

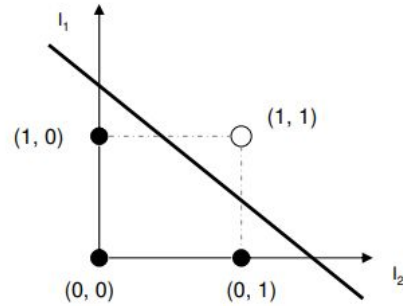


Converting Non-Linear Problem to a Linear Problem

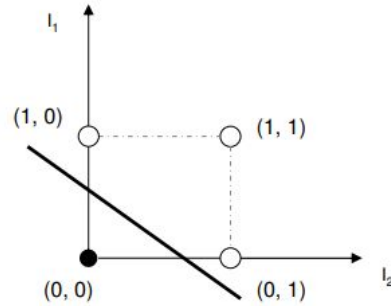


Logic

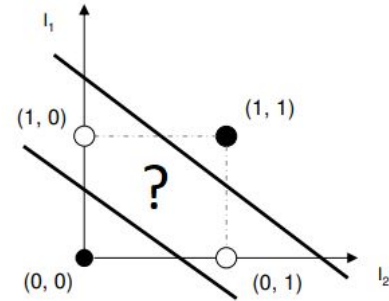
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1

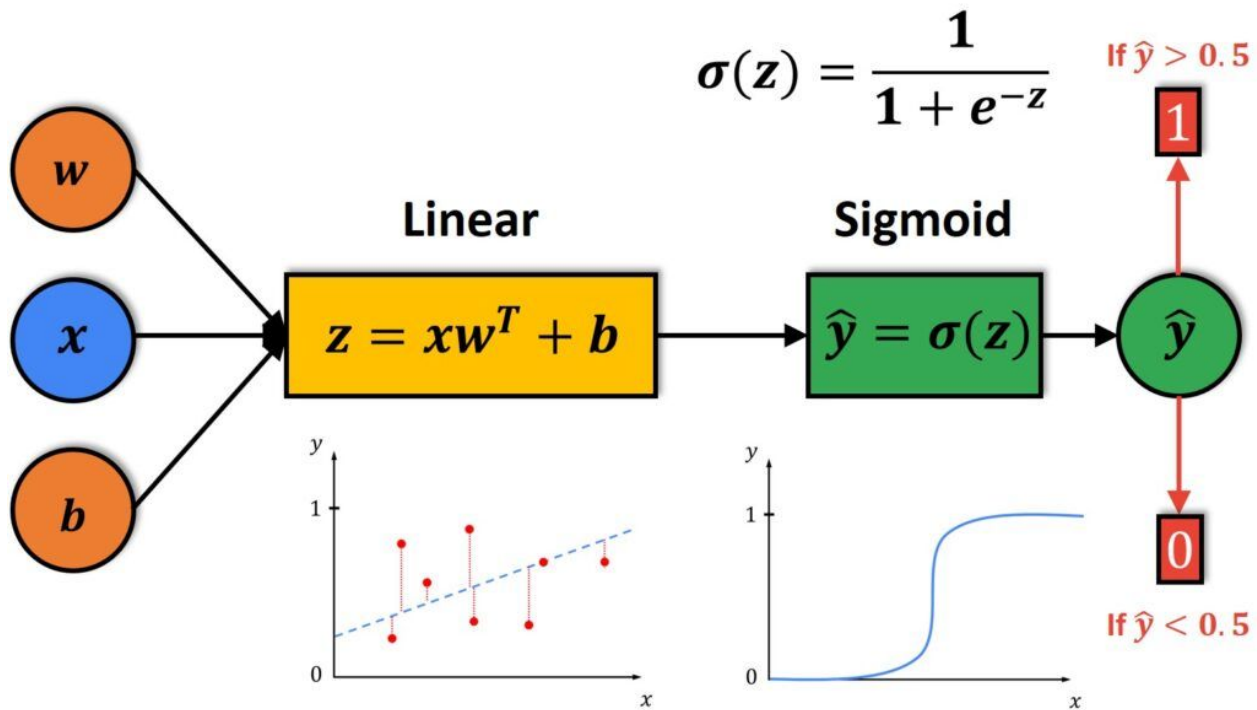


OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1

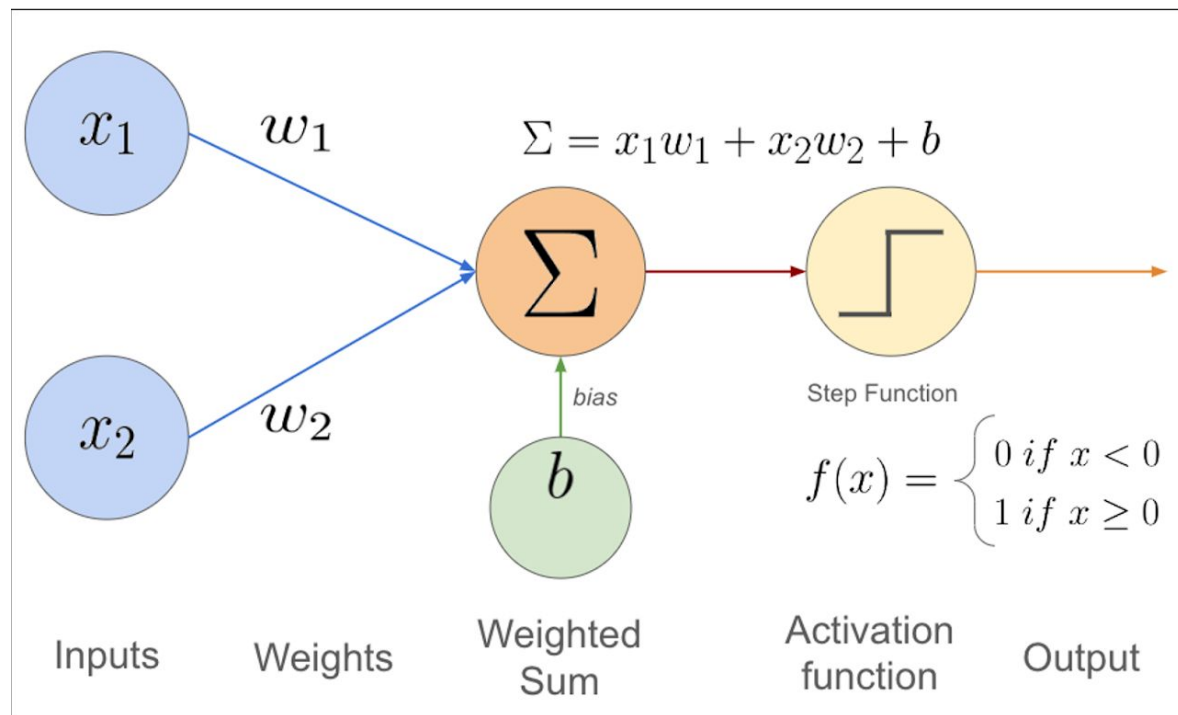


XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0

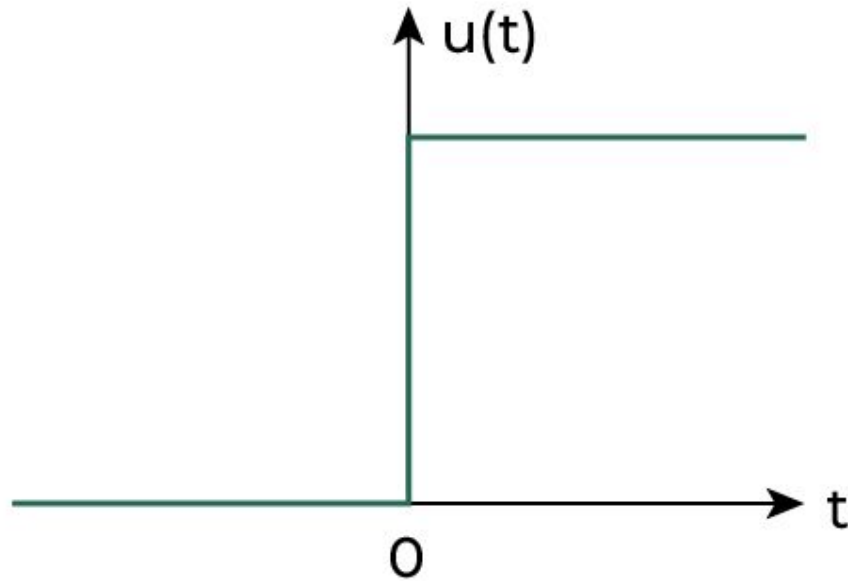




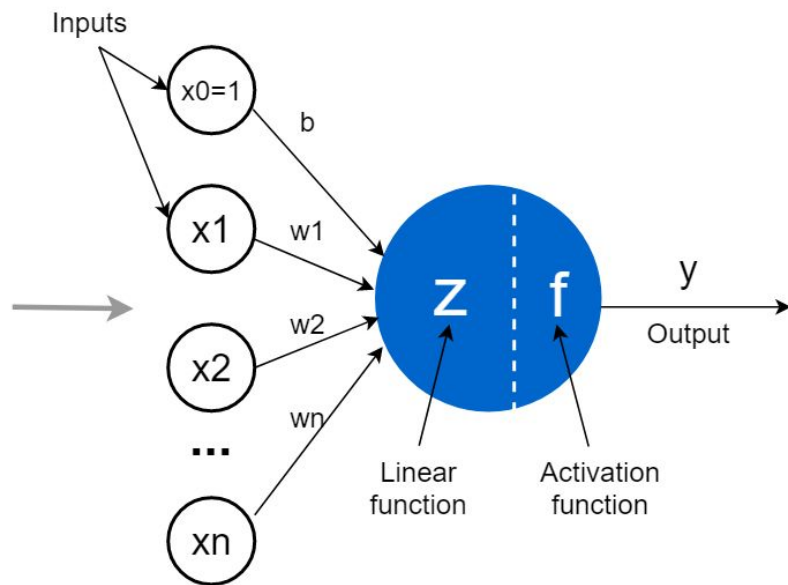
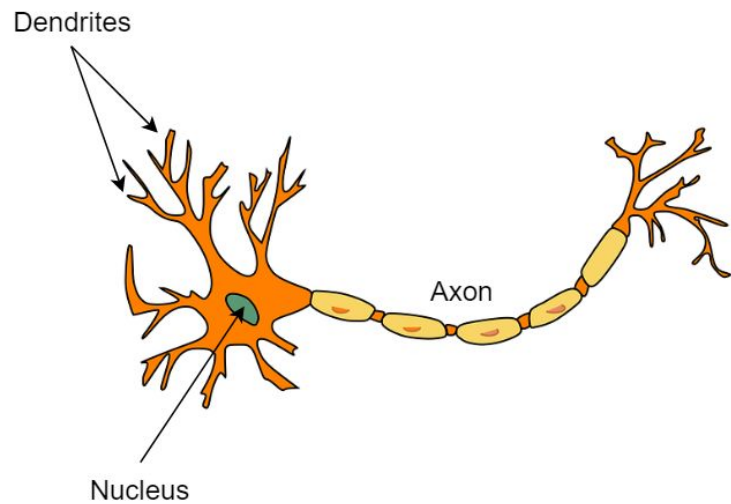
Perceptron

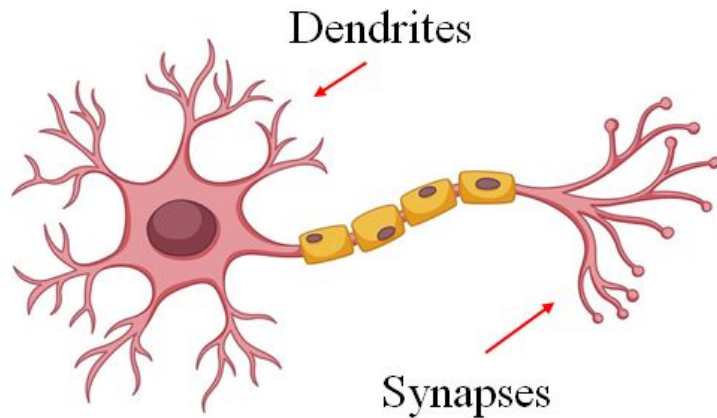


Step Function

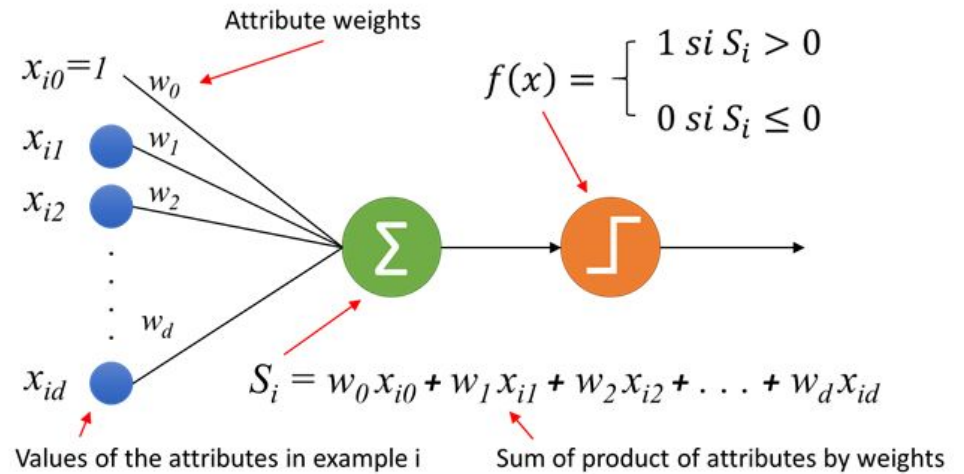


$$u(t) = \begin{cases} 1 & t > 0 \\ 0 & t < 0 \end{cases}$$





NEURON

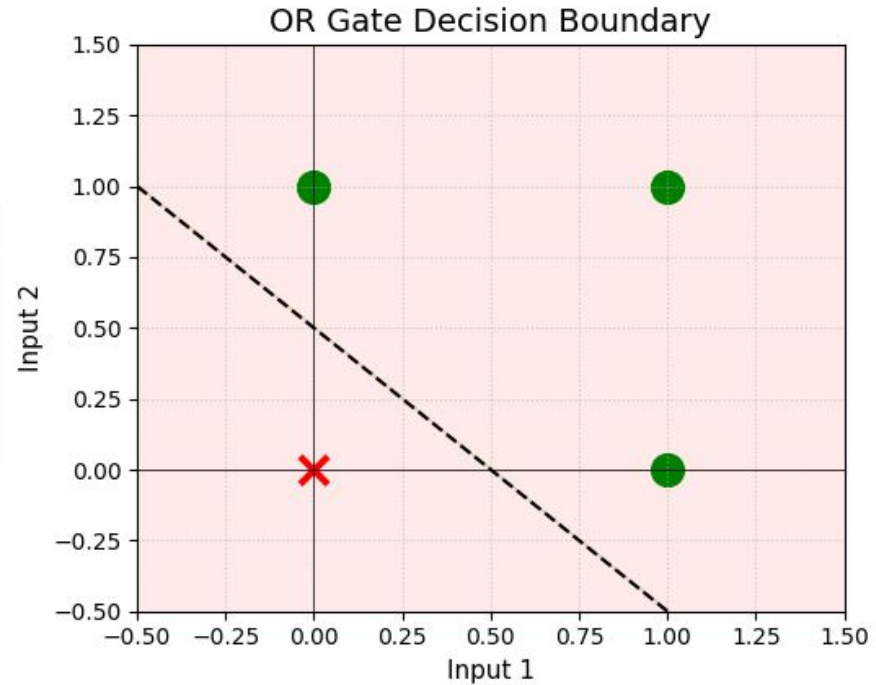


PERCEPTRON

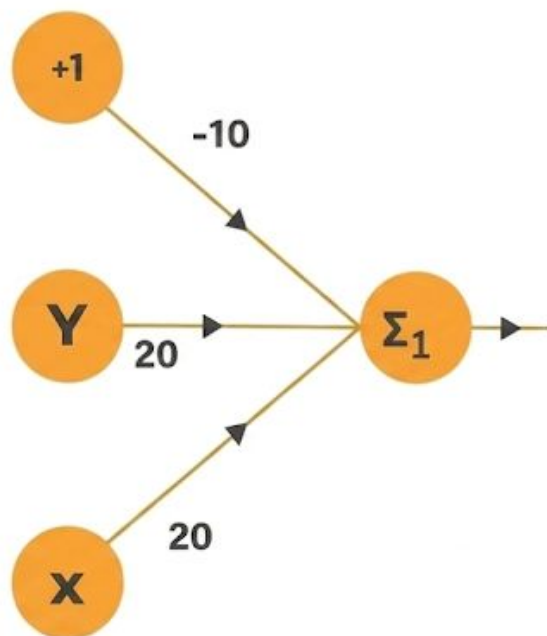
OR Gate

OR Gate Truth Table

Input 1	Input 2	Output (OR)
0	0	0
0	1	1
1	0	1
1	1	1



OR Gate



Inputs: X, Y

Weights: $W_1 = W_2 = 20$

Bias: $b = -10$

Output Equation: $Z = W_1 \cdot X + W_2 \cdot Y + b$

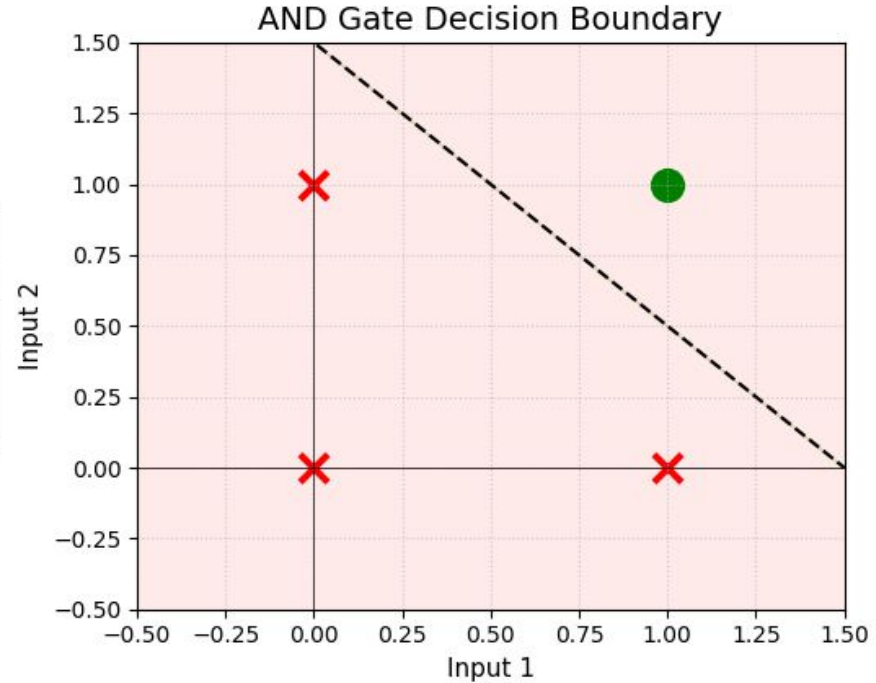
$f(Z) = 1$ if $Z > 0$, else 0

X	Y	$Z = 20X + 20Y - 10$	Step Output $f(Z)$
0	0	$20(0) + 20(0) - 10 = -10$	0
0	1	$20(0) + 20(1) - 10 = 10$	1
1	0	$20(1) + 20(0) - 10 = 10$	1
1	1	$20(1) + 20(1) - 10 = 30$	1

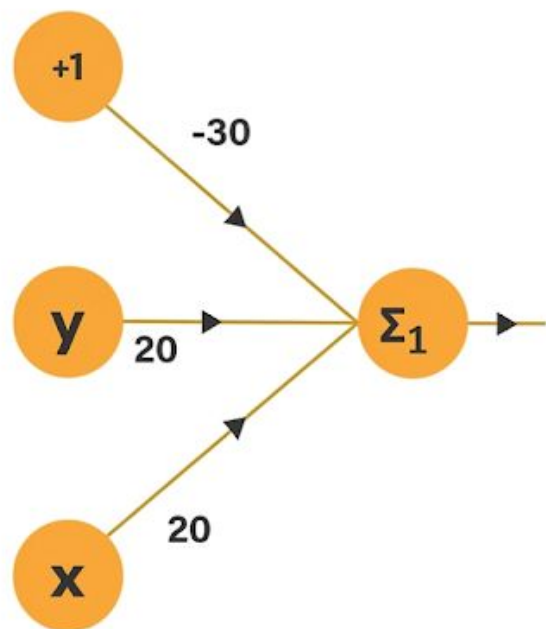
AND Gate

AND Gate Truth Table

Input 1	Input 2	Output (AND)
0	0	0
0	1	0
1	0	0
1	1	1



AND Gate



Inputs: X, Y

Weights: $W_1 = W_2 = 20$

Bias: $b = -30$

Output Equation: $Z = W_1 \cdot X + W_2 \cdot Y + b$

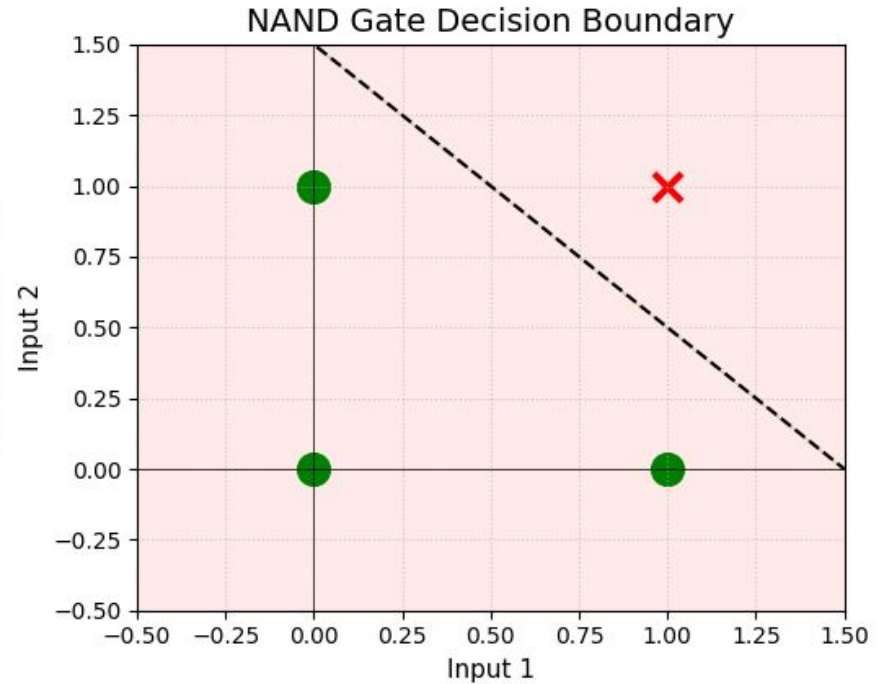
$f(Z) = 1$ if $Z > 0$, else 0

X	Y	$Z = 20X + 20Y - 30$	Step Output $f(z)$
0	0	$20(0) + 20(0) - 30 = -30$	0
0	1	$20(0) + 20(1) - 30 = -10$	0
1	0	$20(1) + 20(0) - 30 = -10$	0
1	1	$20(1) + 20(1) - 30 = 10$	1

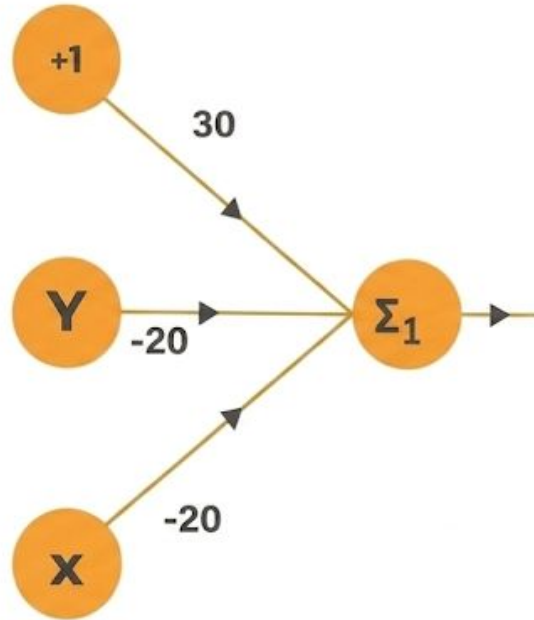
NAND Gate

NAND Gate Truth Table

Input 1	Input 2	Output (NAND)
0	0	1
0	1	1
1	0	1
1	1	0



NAND Gate



Inputs: X, Y

Weights: $W_1 = W_2 = -20$

Bias: $b = 30$

Output Equation: $Z = W_1 \cdot X + W_2 \cdot Y + b$

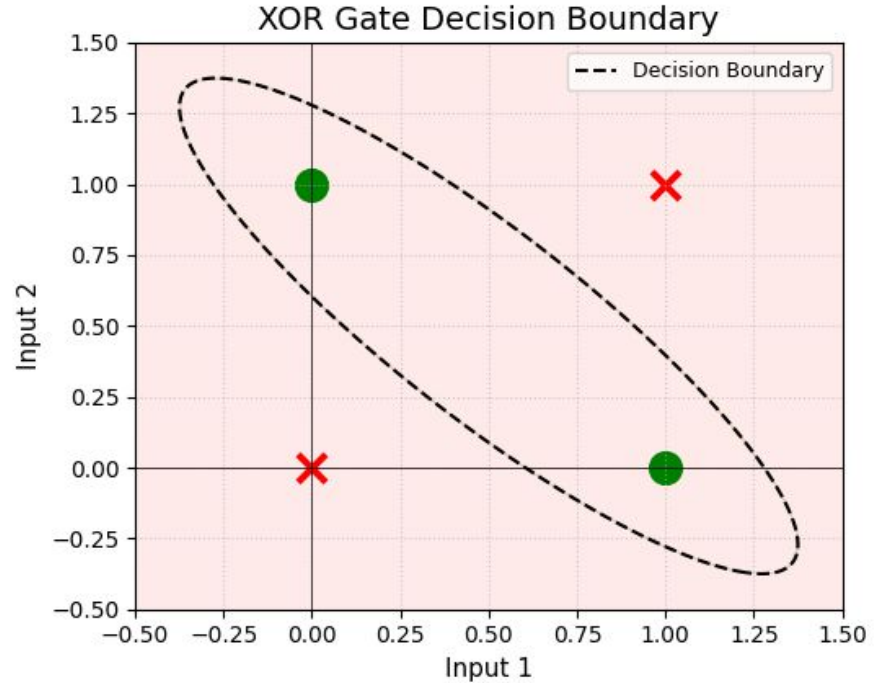
$f(Z) = 1$ if $Z > 0$, else 0

X	Y	$Z = -20X - 20Y + 30$	Step Output $f(Z)$
0	0	$-20(0) - 20(0) + 30 = 30$	1
0	1	$-20(0) - 20(1) + 30 = 10$	1
1	0	$-20(1) - 20(0) + 30 = 10$	1
1	1	$-20(1) - 20(1) + 30 = -10$	0

XOR Gate

XOR Gate Truth Table

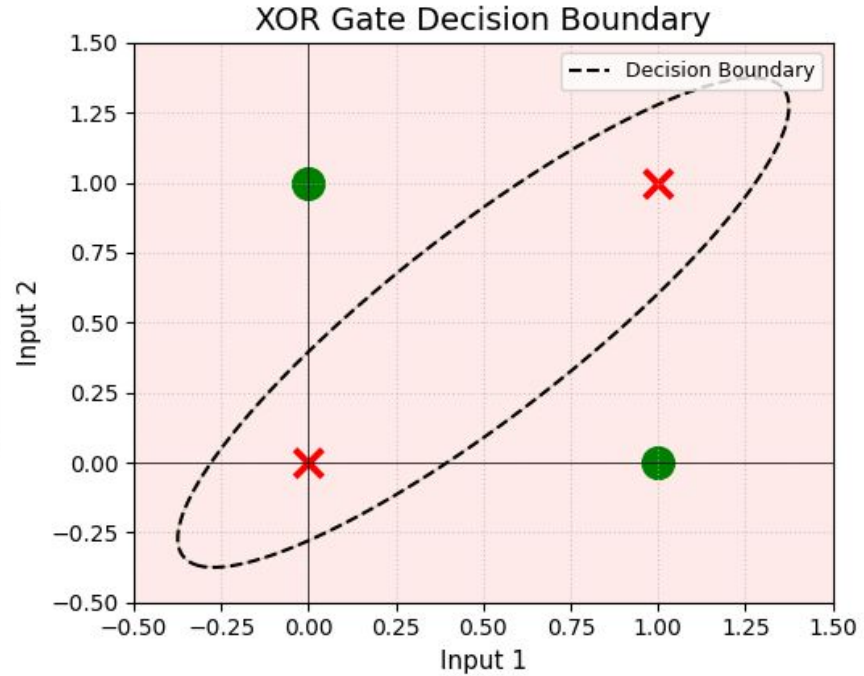
Input 1	Input 2	Output (XOR)
0	0	0
0	1	1
1	0	1
1	1	0



XOR Gate

XOR Gate Truth Table

Input 1	Input 2	Output (XOR)
0	0	0
0	1	1
1	0	1
1	1	0



XOR Gate Implementation Using Basic Gates

Let the two inputs be A and B .

Step 1: The OR Gate

The output of the first step is simply the logical addition of the inputs:

$$\text{Output}_1 = A + B$$

Step 2: The NAND Gate

The NAND gate is the inversion of an AND gate ($\cdot$ representing AND, and the bar representing NOT):

$$\text{Output}_2 = \overline{A \cdot B}$$

Step 3: AND the results together

Now, we pass Output_1 and Output_2 through a final AND gate to get the ultimate XOR output (Y):

$$Y = (A + B) \cdot (\overline{A \cdot B})$$

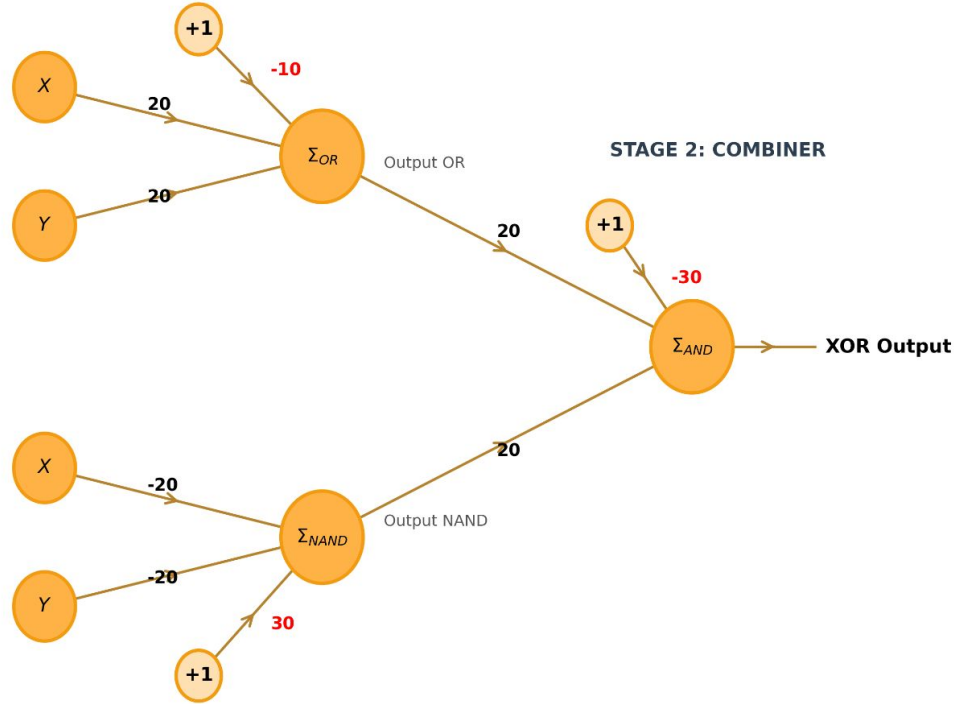
Proof of Equivalence (Why this equals XOR)

We can prove this simplifies to the standard XOR expression $(A\overline{B} + \overline{A}B)$ using De Morgan's Law:

- Apply De Morgan's Law to the NAND part: $\overline{A \cdot B} = \overline{A} + \overline{B}$
- Substitute it back into the equation: $Y = (A + B) \cdot (\overline{A} + \overline{B})$
- Distribute (FOIL) the terms: $Y = A\overline{A} + A\overline{B} + B\overline{A} + B\overline{B}$
- Since $A\overline{A} = 0$ and $B\overline{B} = 0$, we are left with the final standard form:

$$Y = A\overline{B} + \overline{A}B$$

STAGE 1: INDEPENDENT PERCEPTRONS



STAGE 2: COMBINER

AND Combiner Parameters:

Inputs: Output OR, Output NAND

Weights: $W_1 = 20$, $W_2 = 20$

Bias: $b = -30$

$Z_{AND} = 20(OR) + 20(NAND) - 30$

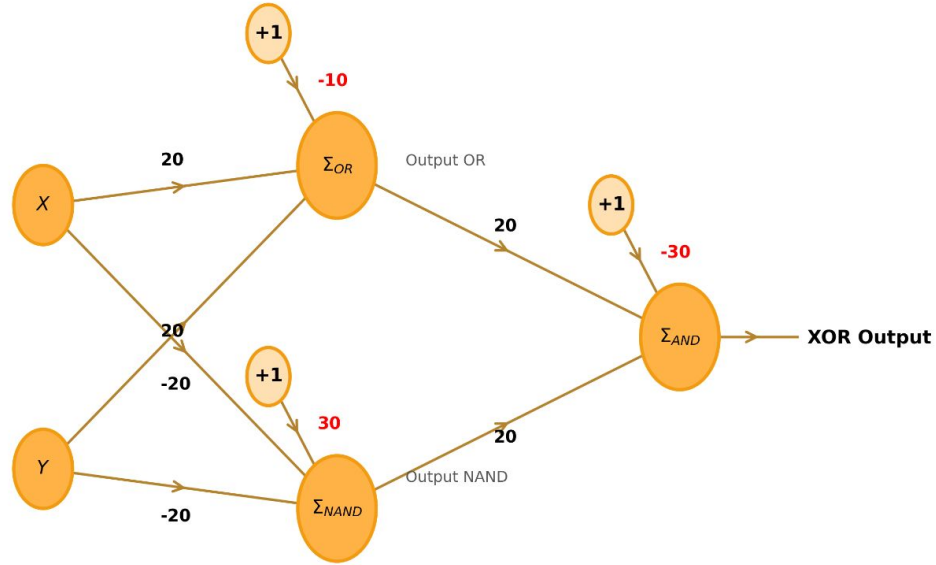
XOR Logic Truth Table:

X	Y	Out OR	Out NAND	Z_AND	f(Z)
0	0	0	1	-10	0
0	1	1	1	10	1
1	0	1	1	10	1
1	1	1	0	-10	0

INPUT LAYER

HIDDEN LAYER

OUTPUT LAYER



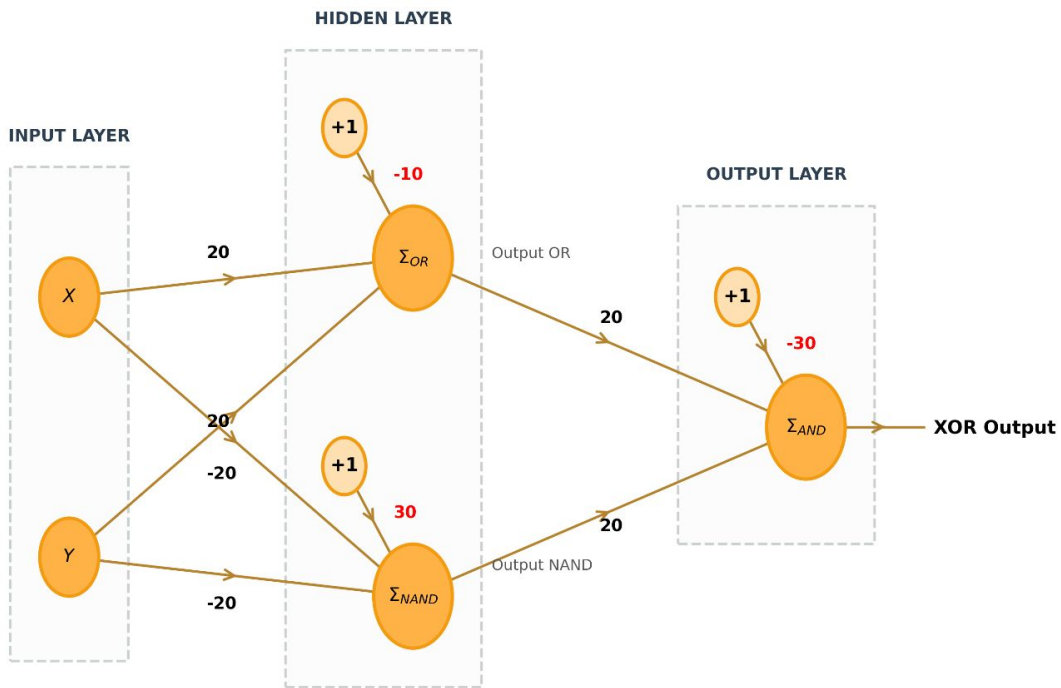
XOR Neural Network:

Hidden Layer: OR & NAND Perceptrons

Output Layer: AND Combiner

XOR Logic Truth Table:

X	Y	Out OR	Out NAND	Z_AND	f(Z)
0	0	0	1	-10	0
0	1	1	1	10	1
1	0	1	1	10	1
1	1	1	0	-10	0



XOR Neural Network:

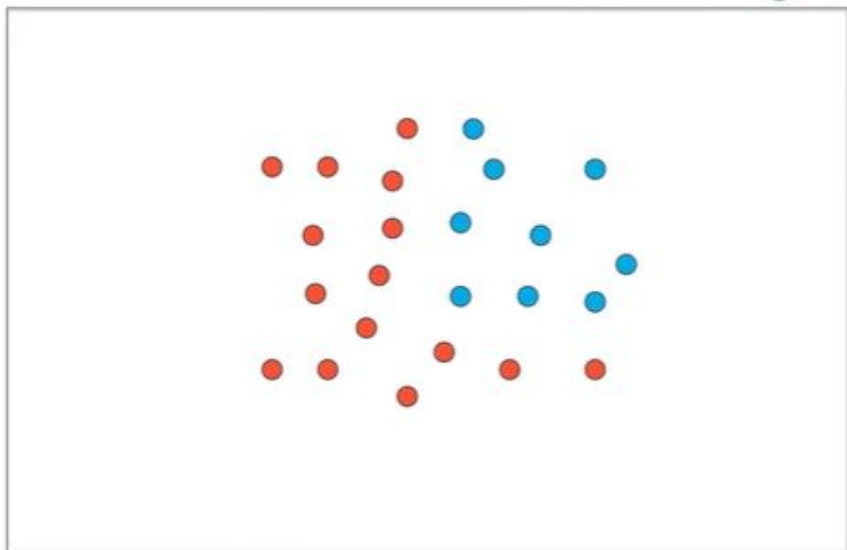
Hidden Layer: OR & NAND Perceptrons

Output Layer: AND Combiner

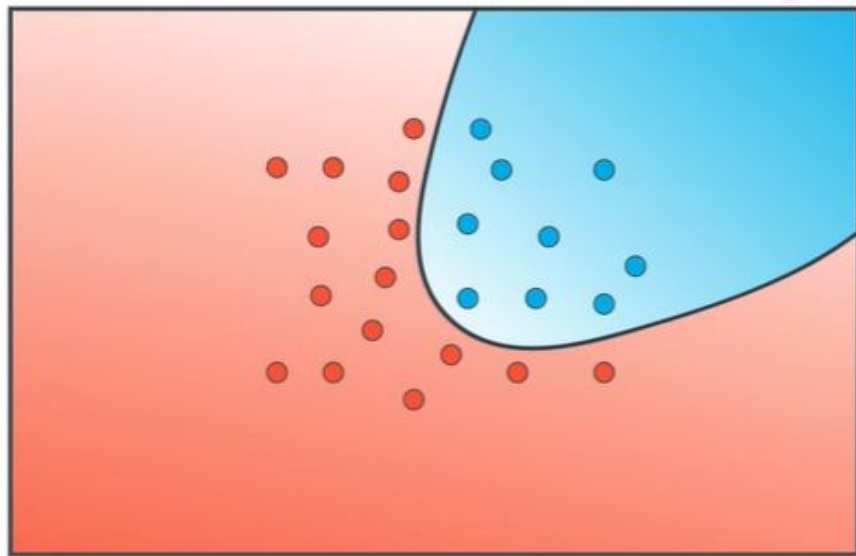
XOR Logic Truth Table:

X	Y	Out OR	Out NAND	Z_AND	f(Z)
0	0	0	1	-10	0
0	1	1	1	10	1
1	0	1	1	10	1
1	1	1	0	-10	0

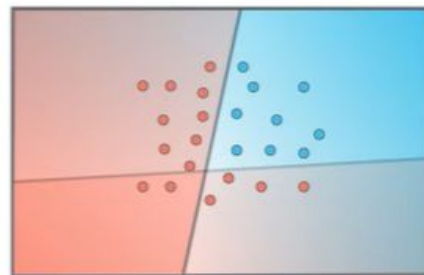
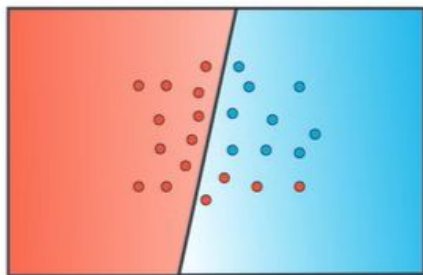
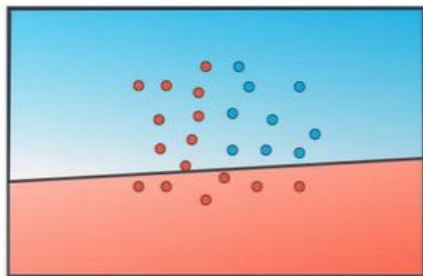
Non-linear regions



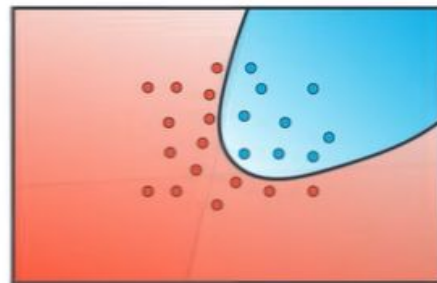
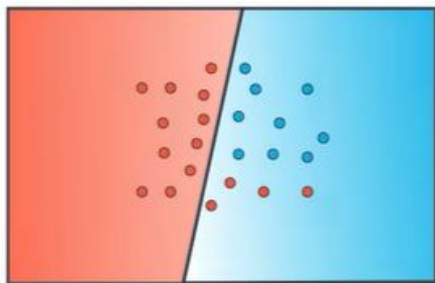
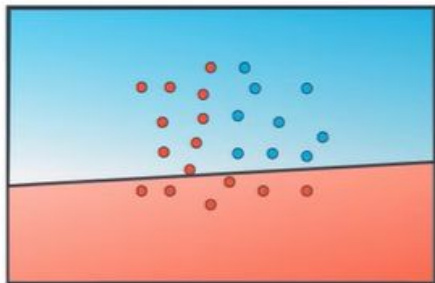
Non-linear regions



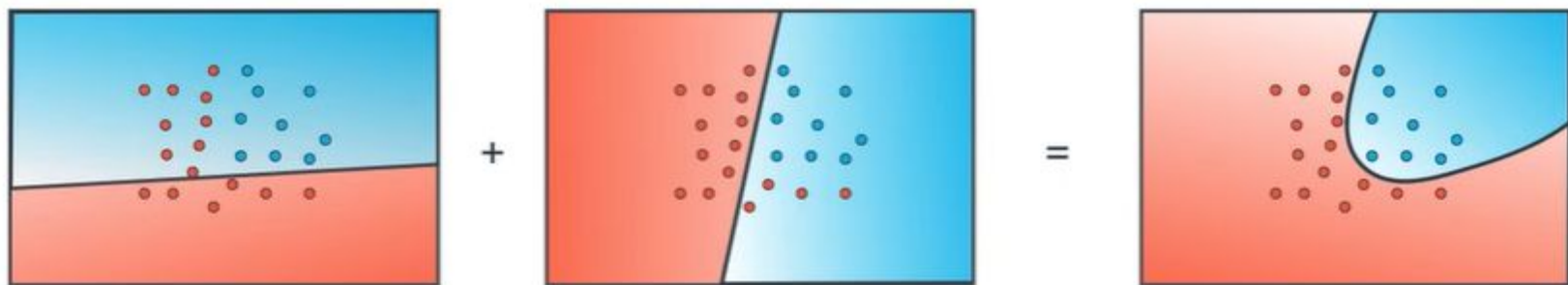
Combining Regions

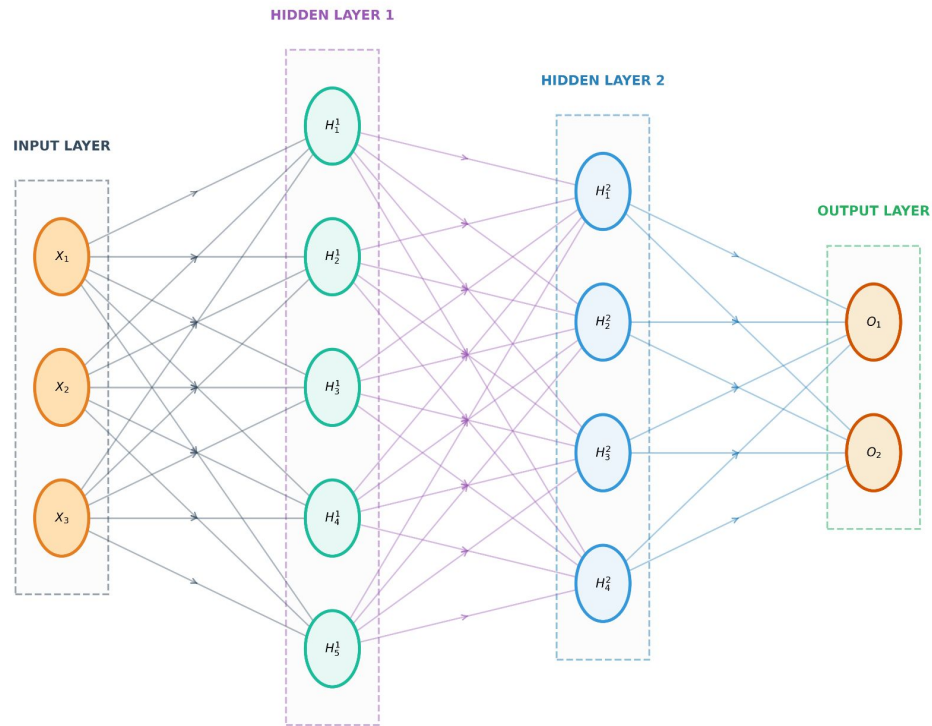


Combining Regions



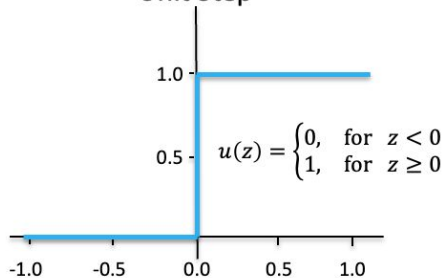
Combining Regions



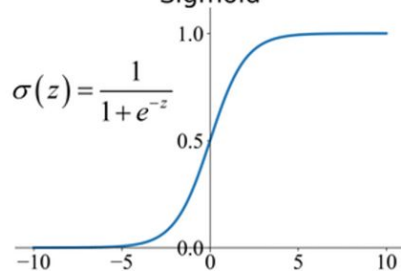


Activation Function: Fire a Neuron

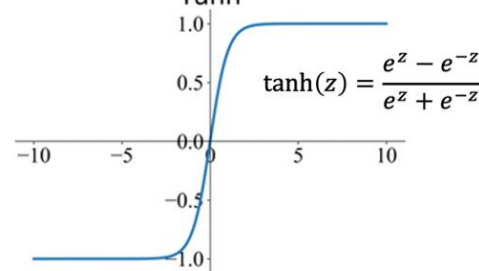
Unit Step



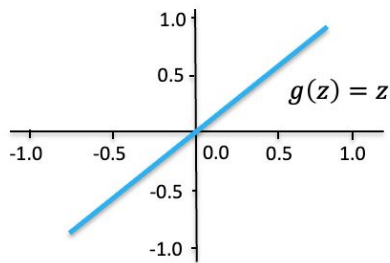
Sigmoid



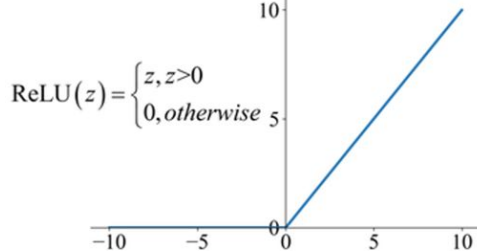
Tanh



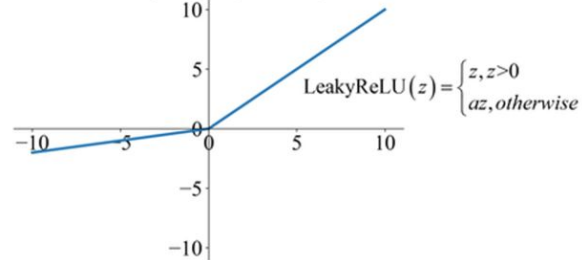
Linear Function

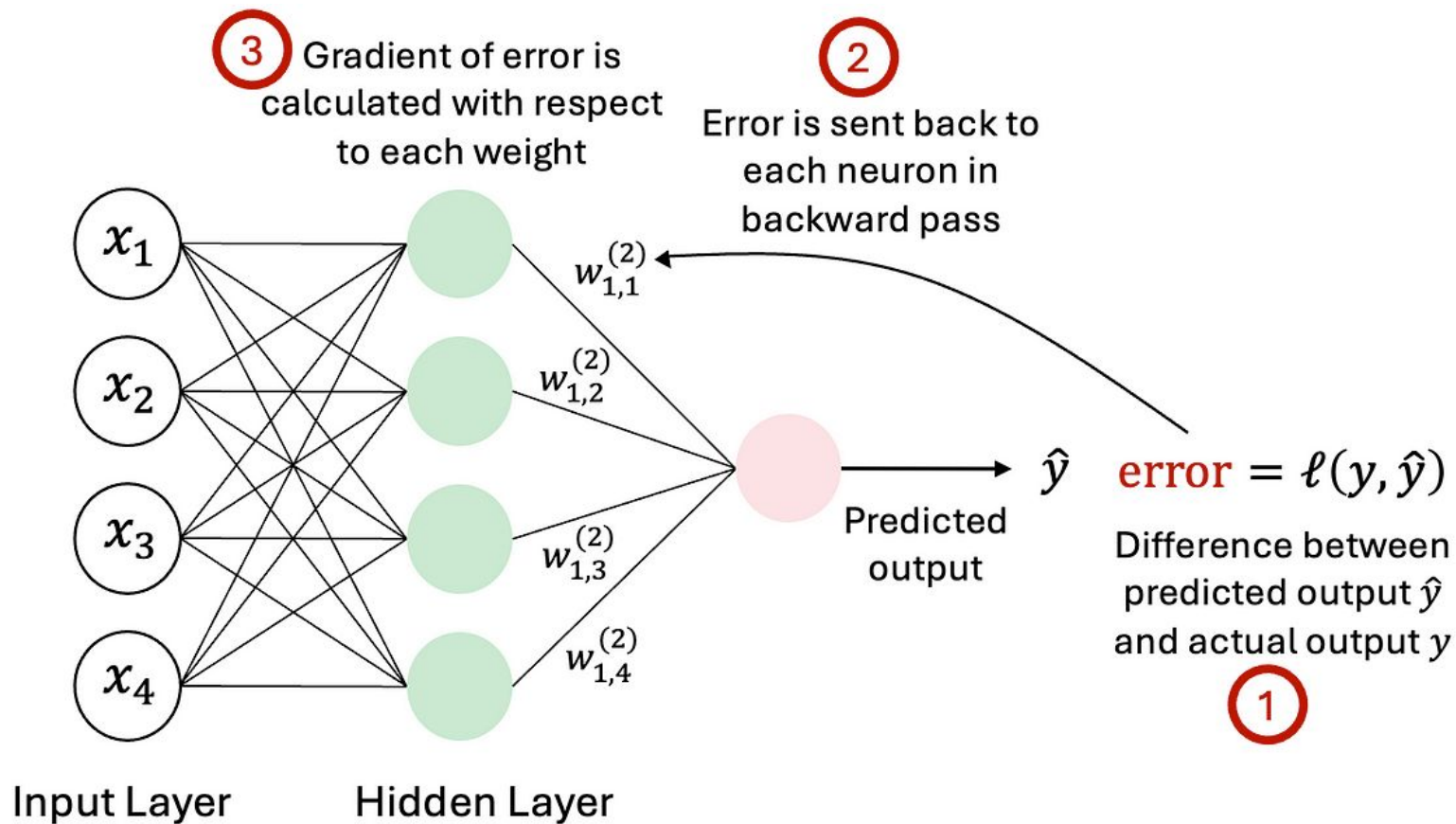


ReLU



LeakyReLU(a=0.2)





Thank You